Credit Risk Modeling with SVMs

Dan Vu

2025-03-12

1 The Problem

Credit card default prediction is a classification problem with a twist: the two kinds of errors are not equally bad. Missing a client who is about to default (false negative) costs the bank money. Flagging a client who is actually fine (false positive) just means a declined transaction which is annoying, but recoverable.

Standard classifiers don't know this. They minimize overall error, which on an imbalanced dataset mostly means learning to predict the majority class and ignoring the minority. We use kernel SVMs to build a credit risk model and explore how to handle this asymmetry explicitly.

Dataset: UCI Default of Credit Card Clients: 30,000 clients, 23 features (payment history, bill amounts, demographics), binary target (default next month: yes/no). No missing values, no preprocessing complications.

2 Data Split

Test set is held out immediately and never touched until the final evaluation. Within the training set, we carve out a validation set for all hyperparameter decisions. Normalization is fit only on train_train and applied forward to val and test.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train_train, X_train_val, y_train_train, y_train_val = train_test_split(X_train, y_train, tes

scaler_X.fit(X_train_train)
X_train_train_scaled = scaler_X.transform(X_train_train)
X_train_val_scaled   = scaler_X.transform(X_train_val)
X_test_scaled        = scaler_X.transform(X_test)
```

```
print(f'train_train: {len(X_train_train_scaled)}')
print(f'train_val:   {len(X_train_val_scaled)}')
print(f'test:        {len(X_test_scaled)}')
```

```
x_train_train_scaled 16800
x_train_val_scaled 4200
x_test_scaled 9000
```

3 Base RBF SVC

RBF SVM is a natural starting point for tabular classification. The kernel $K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2)$ maps the data into a high-dimensional space where a linear separator often exists even when the original feature space isn't linearly separable.

Before tuning anything, let's look at what the default model actually learned. The SVM dual problem is:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$. Points with $\alpha_i > 0$ are support vectors – the only training points that matter at inference time. Prediction for a new point x is:

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

```
from sklearn import svm

rbf_svc = svm.SVC(kernel='rbf', probability=True).fit(X_train_train_scaled, y_train_train)

print('Support vectors per class:', rbf_svc.n_support_)
print('dual_coef_ shape:         ', rbf_svc.dual_coef_.shape)
```

```
Support vectors per class: [4150 3262]
dual_coef_ shape:          (1, 7412)
```

7,412 support vectors out of 16,800 training points which is about 44%. That's high, and it tells us the default decision boundary is not clean. The classes overlap significantly in feature space, so the model needs a lot of points to define the margin.

`dual_coef_` has shape `(1, 7412)` which means one $\alpha_i y_i$ per support vector. A large magnitude means that support vector is pulling hard on the boundary. Most are clustered near the margin; the high-magnitude ones are the contested, borderline clients.

4 Hyperparameter Tuning

Two knobs matter for the RBF SVM:

- **C**: penalty for margin violations. Low C gives a wider, softer margin. High C fits tighter and risks overfitting.
- **gamma**: kernel bandwidth. Low gamma = each point influences a wide region (smooth boundary). High gamma = influence is very local (complex boundary, more support vectors, risks memorization).

We sweep a grid and track both F1 on the validation set and support vector count.

```
from sklearn.svm import SVC
from sklearn.metrics import f1_score

C_vals     = [0.1, 1, 10, 100]
gamma_vals = [0.001, 0.01, 0.1, 1]
results    = []

for C in C_vals:
    for gamma in gamma_vals:
        model = SVC(kernel='rbf', C=C, gamma=gamma)
        model.fit(X_train_train_scaled, y_train_train)
        y_pred = model.predict(X_train_val_scaled)
        f1     = f1_score(y_train_val, y_pred)
        num_sv = model.n_support_.sum()
        results.append((C, gamma, f1, num_sv))

best = max(results, key=lambda x: x[2])
print(f'Best params: C={best[0]}, gamma={best[1]}')
print(f'Best F1:     {best[2]:.4f}')
```

```
Best params: C=1, gamma=0.1
Best F1:     0.4799
```

```
import pandas as pd

df = pd.DataFrame(results, columns=['C', 'gamma', 'F1', 'Support_Vectors'])
df[['C','gamma','F1','Support_Vectors']] = df[['C','gamma','F1','Support_Vectors']].apply(pd.to
df.sort_values(['C','gamma'], inplace=True)

print('F1 Scores')
print(df.pivot(index='C', columns='gamma', values='F1'))
print()
print('Support Vectors')
print(df.pivot(index='C', columns='gamma', values='Support_Vectors'))
```

```
F1 Scores
gamma     0.001     0.010     0.100     1.000
C
0.1    0.000000  0.396605  0.437546  0.015119
1.0    0.219153  0.458424  0.479944  0.264310
10.0   0.406959  0.462990  0.473190  0.292822
100.0  0.466476  0.476923  0.434997  0.298820

Support Vectors
gamma  0.001  0.010  0.100  1.000
C
0.1     7559   7299   8226  13263
1.0     7461   7113   8233  13203
10.0    7247   6983   8338  12615
100.0   7053   6977   8105  11890
```

The sweet spot is C=1, gamma=0.1. We got a few things worth noting:

At gamma=1, support vector count explodes past 11,000 regardless of C. The kernel is so narrow that almost every point needs to be a support vector to define the boundary. F1 collapses too. That's overfitting.

At gamma=0.001, the boundary is so smooth that for C=0.1 the model just predicts the majority class and F1 for the default class hits zero. Too much regularization in both directions.

The C=1, gamma=0.1 combo gives the best F1 while keeping support vector count reasonable at 8,233.

5 Asymmetric Margins

The baseline model at C=1, gamma=0.1 has recall of 0.36 on the default class which means it's missing 64% of actual defaulters. For a bank, that's the worst possible failure mode.

Standard SVM penalizes misclassification equally for both classes. We can fix this by using different penalty parameters C^+ and C^- in the primal:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C^+ \sum_{i:y_i=1} \xi_i + C^- \sum_{i:y_i=-1} \xi_i$$

Setting $C^+ > C^-$ makes misclassifying a defaulter more expensive. The boundary shifts away from the positive class to give it more margin, trading some precision for recall on the class that actually matters.

We try three setups: baseline (no weighting), balanced (auto-computed from class frequencies), and manual ({0:1, 1:5}).

```python
from sklearn import svm
from sklearn.metrics import confusion_matrix, classification_report

best_C, best_gamma = 1, 0.1

baseline = svm.SVC(kernel='rbf', gamma=best_gamma, C=best_C, probability=True)
baseline.fit(X_train_train_scaled, y_train_train)
y_pred_base = baseline.predict(X_train_val_scaled)

balanced = svm.SVC(kernel='rbf', gamma=best_gamma, C=best_C, class_weight='balanced', probabili
balanced.fit(X_train_train_scaled, y_train_train)
y_pred_bal = balanced.predict(X_train_val_scaled)

manual = svm.SVC(kernel='rbf', gamma=best_gamma, C=best_C, class_weight={0:1, 1:5}, probability
manual.fit(X_train_train_scaled, y_train_train)
y_pred_man = manual.predict(X_train_val_scaled)

for label, pred in [('BASELINE', y_pred_base), ('BALANCED', y_pred_bal), ('MANUAL {0:1, 1:5}',
    print(label)
    print(confusion_matrix(y_train_val, pred))
    print(classification_report(y_train_val, pred))
```

```
BASELINE
[[3120  164]
 [ 575  341]]
              precision    recall  f1-score   support

           0       0.84      0.95      0.89      3284
           1       0.68      0.37      0.48       916

    accuracy                           0.82      4200
   macro avg       0.76      0.66      0.69      4200
weighted avg       0.81      0.82      0.80      4200

BALANCED
[[2736  548]
 [ 382  534]]
              precision    recall  f1-score   support

           0       0.88      0.83      0.85      3284
           1       0.49      0.58      0.53       916

    accuracy                           0.78      4200
   macro avg       0.69      0.71      0.69      4200
weighted avg       0.79      0.78      0.78      4200

MANUAL {0:1, 1:5}
```

```
[[2205 1079]
 [ 266  650]]
              precision    recall  f1-score   support

           0       0.89      0.67      0.77      3284
           1       0.38      0.71      0.49       916

    accuracy                           0.68      4200
   macro avg       0.63      0.69      0.63      4200
weighted avg       0.78      0.68      0.71      4200
```

The tradeoff is visible in the numbers. Baseline recall on defaulters: 0.37. Balanced: 0.58. Manual {0:1, 1:5}: 0.71.

Each step up in recall costs precision the model flags more non-defaulters as risky. Whether that's acceptable depends on the business cost ratio. For a bank that loses, say, 10x more on a default than on a declined transaction, the manual weighting is the right call even though accuracy drops to 0.68.

This is the core reason accuracy is a useless metric here. A model that predicts nobody defaults gets 78% accuracy on this dataset and catches zero defaulters.

6 Probability Scoring via Platt Scaling

Hard class labels are often not what you want in credit risk. A probability of default is more useful since it lets you rank applicants, set thresholds, and integrate with downstream risk systems.

Setting `probability=True` enables Platt scaling. After the SVM is trained, a logistic regression is fit using 5-fold internal CV to map the decision function $f(x)$ to a probability:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(A \cdot f(x) + B)}$$

where A and B are fit by MLE. The decision function measures signed distance to the hyperplane and a large positive value means the model is confident the client defaults, large negative means confident they won't. Platt scaling converts that distance into a calibrated probability.

The internal CV is what makes this reasonable – it prevents the calibration from just memorizing training labels. The result is a score you can actually interpret: `predict_proba(x)[1]` reflects the empirical default rate among clients with similar features.

7 Final Evaluation on Test Set

We use the manual-weighted model (C=1, gamma=0.1, class_weight={0:1, 1:5}) as our final model. This is the first and only time we touch the test set.

```
best_model = svm.SVC(kernel='rbf', gamma=0.1, C=1, class_weight={0:1, 1:5}, probability=True)
best_model.fit(X_train_train_scaled, y_train_train)

y_pred_final = best_model.predict(X_test_scaled)
print(confusion_matrix(y_test, y_pred_final))
print(classification_report(y_test, y_pred_final))
```

```
[[4747 2293]
 [ 594 1366]]
              precision    recall  f1-score   support

           0       0.89      0.67      0.77      7040
           1       0.37      0.70      0.49      1960

    accuracy                           0.68      9000
   macro avg       0.63      0.69      0.63      9000
weighted avg       0.78      0.68      0.71      9000
```

Test results hold; the recall on the default class is 0.70, consistent with what we saw on validation. No surprise collapse, which confirms the validation set decisions generalized.

The model catches 70% of defaulters at the cost of flagging about a third of non-defaulters as risky. Whether you'd deploy this is a different story (don't), but it's a meaningful starting point and a significant improvement over the naive baseline that missed 63% of defaulters.

We have many improvements to make from here: a probability threshold sweep to find the operating point that matches your actual loss ratio, and potentially a richer feature set or kernel choice. But the core machinery: asymmetric loss, Platt-scaled probabilities, is pointing us in the right direction.